

Energy-Aware FPGA Implementation of a Spiking Neural Network with LIF Neurons

Ali Oonwala, Arpit Arun Kumaar, Braden Schulz,
Grady Booth and Sreya Roy Chowdhury

University of Waterloo

Waterloo, ON, Canada

{aaoonwal, aakumaar, b2schulz, gjbooth, s4roycho}@uwaterloo.ca

Abstract—We reproduce a 4096–512–2 leaky-integrate-and-fire spiking neural network for 64×64 collision classification over 25 timesteps and explore routable FPGA mappings of the model. Two unreduced full-network organizations are used as simulation references, while four folded accelerator cores are placed and routed on an Artix UltraScale+ AU25P. When input and weight data are supplied continuously, the 128-lane design achieves 601.35 inferences/s and 63.09 effective GOPS at an estimated 1.165 W, or 1.94 mJ/inference. The source paper’s reported 541 GOPS is consistent with the peak arithmetic rate of a fully fed 4096-wide datapath, but its inference rate and weight-delivery architecture are not reported. Our results show the area-throughput tradeoffs of the folded cores and explain why inference rate and energy per inference should accompany peak GOPS.

I. INTRODUCTION

Ali et al. [1] report an FPGA implementation of a spiking neural network (SNN) for collision classification. Their tabulated values give 1,093 GOPS/W for the SNN and 143 GOPS/W for a binarized convolutional neural network (BCNN), a $7.64\times$ ratio or 86.9% lower energy per counted operation under their convention. Because the two results use different devices, workloads, and undefined operation-count boundaries, we treat this as a source-reported comparison rather than a reproduced energy advantage. Leaky-integrate-and-fire (LIF) neurons accumulate signed input current, apply decay, emit a spike after crossing a threshold, reset their membrane state, and suppress further spikes during a refractory period.

The particular problem that the paper focuses on is that of an automobile’s dashcam, which must predict whether the vehicle is on a collision course based on a grayscale image. While a convolutional neural network would multiply the exact value of each pixel by a certain weight in the convolution, a spiking neural network translates the grayscale value of each pixel into a probability value of a spike; over several time-steps, the number of spikes accurately represents the value of each pixel. The beauty of this is that for each time-step, the model does not need to multiply the inputs by a weight; the weights can instead be multiplexed by the input, which will be either 0 or 1.

II. BACKGROUND AND DESIGN RATIONALE

The main accelerator components of our design are the LIF module and cascaded adder. In the LIF module, we set the

spike output to be combinational, spiking on the same cycle as the updated internal value exceeds the threshold. The cascaded adder sums spike-conditioned weight terms: each binary spike selects its 16-bit weight or zero, and a balanced tree reduces the selected terms. The fully pipelined implementation has a valid latency of $\lceil \log_2(N) \rceil + 1$ cycles and can accept one vector per cycle; the folded implementations instead use narrower combinational or locally pipelined trees. Hardware cost grows linearly with the number of spatial terms.

To connect these modules, we have the main collision_predictor module. When designing this module, we faced some ambiguity in the paper’s original design. Specifically, we were unsure whether the paper implemented a highly-parallelized design where many LIFs were calculated at the same time, or a more area-efficient design where a single LIF was used to sequentially calculate all of the second-layer spiking activity. We also considered a further reduction in area, where the cascaded adder was folded, calculating only 16, 64, or 128 additions within a cascaded system, then accumulating the results over several cycles. This was intended to reduce the power draw and LUT count for the design to a similar level to what was seen in the paper.

III. TESTING METHODOLOGY

Unit and integration tests were created to verify our RTL implementations. Most functional tests use cocotb with Verilator; separate SystemVerilog testbenches generate switching activity for Vivado power analysis.

A. Unit Testing

We developed a suite of unit tests for both the cascaded adder module and lif-neuron module to ensure module-level functionality.

The cascaded adder unit tests covered functional correctness, signed and boundary inputs, pipeline timing, reset behaviour, and robustness when invalid inputs change. The suite was run with 2, 512, and 4096 inputs. The test plan is available at `testing/cascaded_adder/test_plan.md`.

The LIF model was tested for reset behaviour, enable gating, integrate–leak–fire functionality, refractory behaviour, and randomized lockstep against a Python arithmetic reference. Three paired configurations were tested: $(k, u_{th}, u_{rest}, r_{max}) \in$

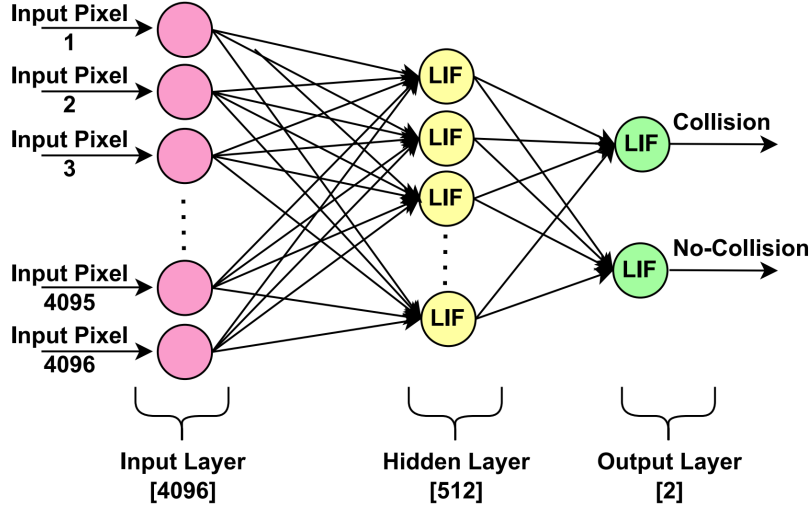


Fig. 1. Model architecture presented by Ali et al. [1] which our work aims to accelerate. The 25 time-steps are not shown in this image. Source: Ali et al. [1].

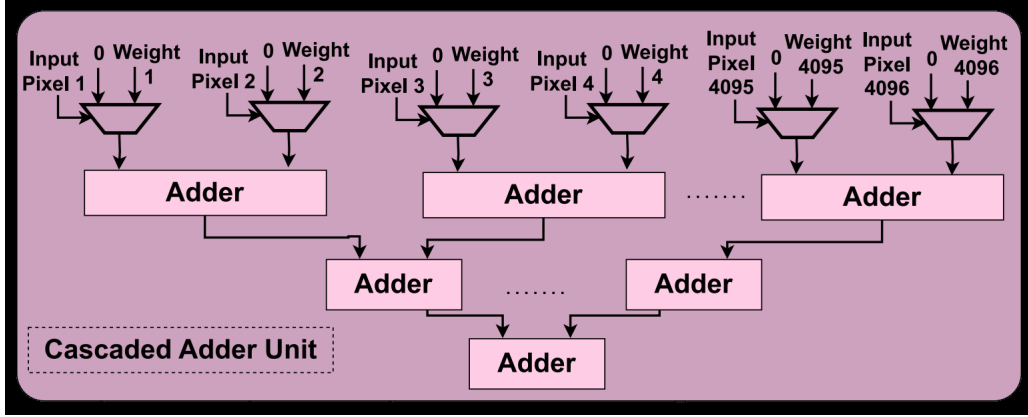


Fig. 2. Cascaded adder unit described by Ali et al. [1]. Source: Ali et al. [1].

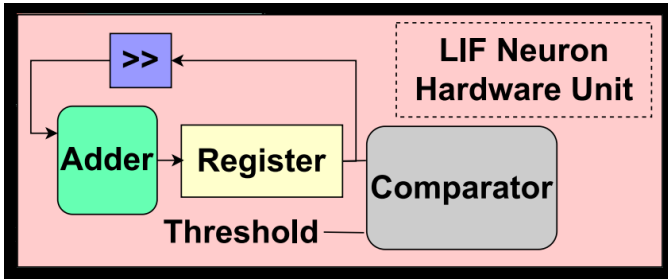


Fig. 3. LIF neuron module described by Ali et al. [1]. Source: Ali et al. [1].

$\{(5, 100, 0, 5), (2, 20, 5, 5), (1, 8, 0, 2)\}$. The test plan is available at `testing/lif_model/test_plan.md`.

B. Integration Testing

The software reference was evaluated on all 1,576 DroNet images, resized to 64×64 grayscale pixels. RTL verification replayed deterministic saved spike traces on smaller

subsets. The sequential design checks the first three images against an integer arithmetic reference and separately checks `out_valid` latency for three images. Folded designs check one or two normal images, while the L128 suite also checks one image with weight-stream backpressure. These tests establish agreement only for the evaluated traces; they do not establish full-test-set RTL accuracy. The fully parallel design is retained as a simulation reference, but a complete 1,576-image regression is not claimed.

IV. WORKLOAD AND SOFTWARE REFERENCE MODEL

A. Software Reference Model

The software reference model leveraged the `snntorch` Python package to train an SNN as described in the paper, using the same DroNet collision-avoidance dataset cited therein. The test set contained 1,576 images. The model converted each image to grayscale and resized it to 64×64 pixels. It encoded each image as a Bernoulli rate-coded spike train with 25 timesteps.

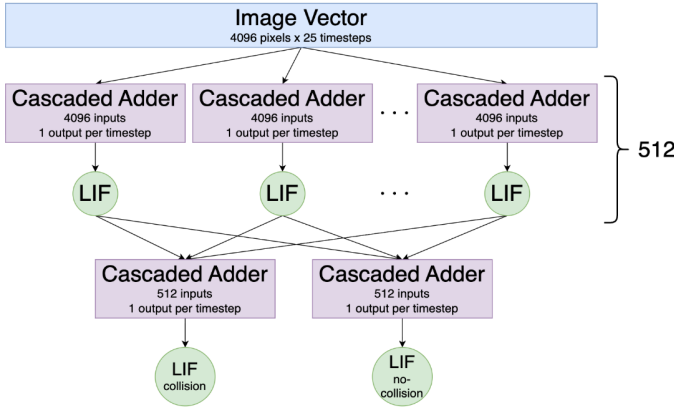


Fig. 4. Hardware architecture for the fully-parallel design (not synthesized).

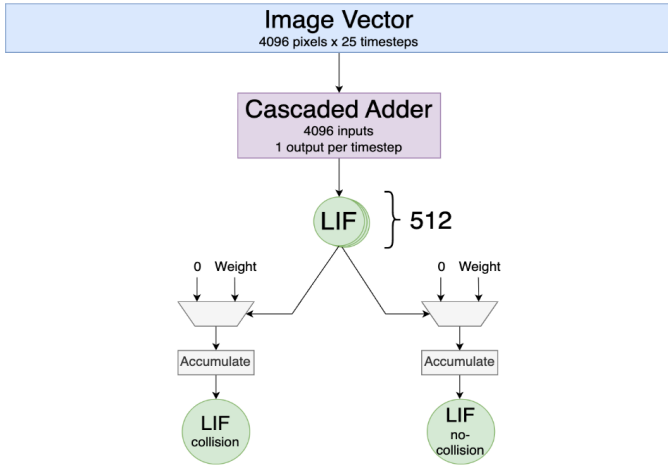


Fig. 5. Hardware architecture for the resource-shared design (not synthesized).

The network was a 4096–512–2 LIF SNN. Exported weights and biases use signed 16-bit Q1.15 representation, while dot-product accumulators are wider. The retained training log reaches 96.43% training accuracy and 82.42% test accuracy; the latter is 2.58 percentage points below the 85% test accuracy reported by Ali et al. for 64×64 inputs [1]. Accuracy is a software classification result and is distinct from RTL trace equivalence.

B. Workloads

1) *Dataset and Verification Workload*: The software reference model was evaluated on the full 1,576 image Dronet test set. RTL functional verification replayed a deterministic subset of the software-generated spike traces. Post-route power analysis used switching activity (SAIF) from one complete trained-image inference.

V. RTL IMPLEMENTATION

We developed two simulation-only full-network reference organizations, four routed folded full-network cores, and one routed out-of-context (OOC) hidden-neuron datapath. Every design uses the same Q1.15 weights, biases, and LIF rule.

They differ in spatial parallelism, scheduling, buffering, and weight delivery.

A. Cascaded Adder & LIF Neuron Hardware Unit

Following the paper, a single neuron is a cascaded adder followed by a LIF neuron hardware unit. Because input spikes are binary, the cascaded adder replaces a multiply-accumulate: each spike selects its 16-bit weight or zero, and a pipelined adder tree reduces those terms. Bias is added after the tree. The LIF unit then updates membrane potential, applies decay and a 5 step refractory period, and emits a binary spike when the potential exceeds the learned threshold. These two blocks are the arithmetic core of every design below.

B. Parallel and Resource-Shared Designs

We first built two full-network RTL models in order to verify the 4096-512-2 mapping independently of FPGA area limits. Weights and biases are preloaded from the software-exported hexadecimal tables. Input spike trains are encoded offline and replayed in simulation. Unfortunately, neither design is placed on the FPGA due to resource/area limitations on available hardware.

The parallel design instantiates 512 hidden-layer cascaded adders and 512 LIF units, then two 512-input cascaded adders and two output LIF units (collision and no-collision). All hidden neurons evaluate a timestep together. This is the throughput-oriented unrolling of the network and an upper bound on full-network parallelism. A 512-wide bank of 4096-input trees, plus the on-chip layer-1 matrix, does not fit the AU25P.

The resource-shared design instantiates one 4096-input cascaded adder and reuses it across the 512 hidden neurons, so each timestep takes 512 neuron cycles. Hidden spikes are folded into the two output classes with a spike-gated accumulate, rather than two 512-input trees. This is the area-oriented full-network golden model. It still does not fit the device: the adder remains a full 4096-input tree, and the layer-1 weights remain on chip.

These two designs are the functional references. They only establish bit-level behavior and a throughput-versus-area bound and are not the source of LUT, FF, BRAM, frequency, or power results.

C. Folded FPGA Designs

This work extends our poster-stage implementation. One 4096-input tree can be routed OOC on the AU25P, but the complete network with replicated trees and/or its 4-MiB layer-1 matrix cannot be placed using the unreduced mappings. The folded cores therefore time-multiplex each 4096-term weighted sum onto a P -lane datapath and stream layer-1 weights. A *lane* evaluates one binary-spike/16-bit-weight interaction per compute cycle; an *engine* is one independently stateful P -lane datapath; and the *fold factor* is $4096/P$. Core names use the form E_m/L_P , where m is the number of engines (omitted when $m = 1$) and P is the number of lanes per engine. Thus L16, L64, and L128 denote one-engine designs with 16, 64, and 128 lanes, respectively, while

E4/L16 denotes four parallel 16-lane engines. All 25 input-spike frames are buffered on chip. Each engine has two resident weight-row buffers so that one row can be prefetched while the other is used.

We placed and routed four folded accelerators:

- L16: one engine with 16 lanes ($P = 16$);
- L64: one engine with 64 lanes ($P = 64$);
- L128: one engine with 128 lanes ($P = 128$); and
- E4/L16: four 16-lane engines that advance four hidden neurons together.

The 16 and 64-lane engines, as well as the 4 engine design, accept a 64-bit weight stream (rather than storing weights on chip). The 128-lane design accepts a 128-bit stream and reduces each fold with eight pipelined 16-input cascaded adders. Folding here is not the same as the resource-shared design which has a 4096-input tree. The folded designs narrow that tree to P lanes so that a full inference fits the FPGA.

To compare a paper-style datapath without placing the full network, we also implement one cascaded adder and LIF neuron hardware unit out of context: a single 4096-input tree plus one LIF. That partition measures the paper’s per-neuron arithmetic. It is not a 4096-512-2 FPGA mapping.

LUT, FF, BRAM, frequency, and power estimates in later sections come only from the folded accelerators and the OOC unit. Cycle-accurate checks of the unreduced mapping use the parallel and resource-shared simulation models.

VI. RESULTS AND DISCUSSION

A. Metric Definitions

One *inference* is one complete 25-timestep image classification, including the final comparison of the two output spike counts. The network evaluates

$$N_{\text{syn}} = 25[(4096 \times 512) + (512 \times 2)] = 52,454,400$$

synaptic interactions per inference. We count weight selection and accumulation as two operations, so $N_{\text{op}} = 104,908,800$; a one-operation convention halves every GOPS value. Peak GOPS is $2 \times \text{engines} \times \text{lanes} \times f_{\text{clk}}$. Effective core GOPS is $N_{\text{op}}(f_{\text{clk}}/C_{\text{inf}})/10^9$.

We report ideal-stream core inferences/s as $f_{\text{clk}}/C_{\text{inf}}$. The cycle count includes core input loading, accepted weight beats, folding, LIF updates, and control, but the testbench supplies continuous input and weight streams. It excludes preprocessing, DDR/AXI controller stalls, and external-memory power; it is therefore not deployed system throughput. The L128 core reaches 601.35 inferences/s at an estimated 1.165 W, or 1.94 mJ/inference. Ali et al. do not report an inference rate, so their energy per inference cannot be reconstructed.

B. Interpreting the Published GOPS

The published 541 GOPS is close to $4096 \times 2 \times 67 \text{ MHz} = 548.9 \text{ GOPS}$, suggesting a peak-rate calculation for a fully fed 4096-wide datapath. This interpretation is plausible but does not establish completed-network throughput. Sustaining 541 GOPS requires approximately 4,037 synaptic interactions

per cycle. If a new 16-bit weight row were required every cycle, this corresponds to about 541 GB/s; even reusing each row across all 25 timesteps would imply roughly 21.6 GB/s at the corresponding full-network inference rate. The source paper reports neither the schedule nor the weight-delivery path.

The layer-1 matrix contains $4096 \times 512 \times 16$ bits, or 4 MiB. Ali et al. report neither BRAM use nor an external interface. Their complete-network row lists 6,521 LUTs and 1,780 registers, whereas our OOC implementation of only one straightforward 4096-input tree and LIF uses 38,667 LUTs and 69,606 FFs at 67 MHz. Because the original part number, netlist, folding schedule, and memory organization are unavailable, we cannot reconcile its reported area and peak throughput from the disclosed architecture.

TABLE I
PUBLISHED SNN VERSUS OOC COMPUTE UNIT

Metric	Published	OOC 67	OOC 280
FPGA	Artix-7	AU25P	AU25P
Freq. (MHz)	67	67	280
Power (W)	0.495	1.332	3.874
Peak GOPS	541	548.9	2294
Peak GOPS/W	1093	412	592
LUTs	6,521	38,667	38,713
FFs	1,780	69,606	69,606

Published values use the source paper’s reporting boundary. OOC is one fully fed hidden-neuron datapath, not a full-network accelerator.

TABLE II
PUBLISHED RESULT VERSUS L64 ROUTED CORE

Metric	Published	L64 core
Clock (MHz)	67	125
Inferences/s	Not reported	149.49
Effective GOPS	541 reported	15.68
LUT / FF	6,521 / 1,780	5,130 / 3,348
BRAM36 equiv.	Not reported	34.5
Power (W)	0.495	0.581
Effective GOPS/W	1,093	26.99

TABLE III
FOLDED-CORE RESULTS SUMMARY

Design	MHz	Inf./s	GOPS	LUT	W
E4/L16	160	191.27	20.07	5,017	0.621
L16	175	53.06	5.57	2,285	0.509
L64	125	149.49	15.68	5,130	0.581
L128	250	601.35	63.09	11,482	1.165

Rates assume continuous input and weight streams. Power is a workload-dependent SAIF estimate and excludes DDR, AXI, controller, and preprocessing power.

VII. CONCLUSION & FUTURE WORK

We developed two unreduced simulation references and four routed folded full-network cores for the 4096–512–2 SNN. The routed designs expose a clear area–latency tradeoff: L16 minimizes LUT use and total power, while L128 achieves the highest ideal-stream core throughput and lowest estimated energy per inference. However, differences in device, reporting boundary, operation count, and the undocumented source memory path prevent direct validation of the energy-efficiency claim in [1]. Our main reproducibility result is therefore a

quantified explanation of the architectural information required to interpret the reported GOPS.

Future work should target an FPGA with sufficient logic and on-chip memory to synthesize the complete design and retain all weights on chip, eliminating repeated DRAM-to-BRAM transfers. This would enable a shorter, more controlled study of full-network throughput, power, and resource use.

ARTIFACT AVAILABILITY

<https://git.uwaterloo.ca/ece493-s26/projects/team-22>

REFERENCES

- [1] Asmer Hamid Ali, Mozghan Navardi, and Tinoosh Mohsenin, "Energy-Aware FPGA Implementation of Spiking Neural Network with LIF Neurons," arXiv:2411.01628, 2024.